MATERIA

INF761

Desarrollo de Videojuegos

Guía de Laboratorio 6

Platformer II: Vida al Personaje — Animaciones 2D y Movimiento Pulido

Carrera: Ingeniería Informática

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Semestre: I-2026

Universidad: Universidad Autónoma Tomás Frías (UATF)

Objetivos de Aprendizaje

1. Importar y configurar sprite sheets para crear Animation Clips del personaje (Idle, Run, Jump, Fall).
2. Construir un Animator Controller con transiciones basadas en parámetros (Speed, isGrounded, yVelocity) para controlar las animaciones del personaje.
3. Refactorizar el controlador de movimiento de la Guía 5, incorporando técnicas de game feel: Coyote Time, Jump Buffering y salto de altura variable.
4. Integrar el sistema de animaciones con el controlador de movimiento para lograr un personaje responsivo y fluido.

Prerrequisitos

- Guía 5 completada: proyecto PlatformerGame con nivel de tilemap funcional, Player con movimiento básico, y Cinemachine configurado.
- Sprite sheet del personaje con al menos 4 estados: Idle, Run, Jump, Fall (se recomienda Pixel Adventure 1 de Pixel Frog).
- Unity 6 LTS con el proyecto PlatformerGame abierto.
- Comprensión básica de máquinas de estado (cada animación es un estado; las transiciones se activan por condiciones).

1. Marco Teórico

1.1 Sistema de Animación 2D en Unity

Unity utiliza un sistema de animación basado en tres componentes principales que trabajan juntos: los Animation Clips que definen cada movimiento individual, el Animator Controller que organiza las transiciones entre clips, y el componente Animator que se adjunta al GameObject para reproducir las animaciones.

Componente	Descripción
Sprite Sheet	Imagen que contiene todos los frames de una animación organizados en cuadrícula. Se corta con el Sprite Editor en sprites individuales.
Animation Clip	Archivo .anim que define una secuencia de sprites reproducidos en el tiempo. Cada clip representa un estado: Idle, Run, Jump, Fall.
Animator Controller	Máquina de estados (.controller) que define qué clip se reproduce y cómo transicionar entre ellos según parámetros.
Animator	Componente del GameObject que ejecuta el Animator Controller y aplica las animaciones al SpriteRenderer.

1.2 Transiciones y Parámetros del Animator

Las transiciones entre estados del Animator se controlan mediante parámetros que el código actualiza en cada frame. Los tipos de parámetros disponibles son Float (valores decimales como velocidad), Int (valores enteros), Bool (verdadero/falso como isGrounded), y Trigger (se activa una vez y se resetea automáticamente).

Para un platformer típico, se necesitan al menos estos parámetros y transiciones:

Desde	Hacia	Condición	Significado
`Idle`	`Run`	`Speed > 0.01`	El jugador empieza a moverse
`Run`	`Idle`	`Speed < 0.01`	El jugador se detiene
`Idle/Run`	`Jump`	`isGrounded = false` `yVelocity > 0`	El jugador salta
`Jump`	`Fall`	`yVelocity < 0`	Empieza a caer
`Fall`	`Idle/Run`	`isGrounded = true`	Aterriza en el suelo

Has Exit Time y Transition Duration

Para un platformer responsivo, desactivar Has Exit Time en todas las transiciones. Esto permite que las transiciones ocurran inmediatamente cuando se cumple la condición, sin esperar a que el clip actual termine.

Establecer Transition Duration en 0 para que el cambio de animación sea instantáneo (sin blending entre frames).

1.3 Game Feel: Técnicas de Movimiento Pulido

Un buen platformer se distingue por la sensación de control (game feel). Las siguientes técnicas, utilizadas en juegos clásicos y modernos, mejoran drásticamente la experiencia del jugador sin cambiar la mecánica fundamental:

Coyote Time (Tiempo del Coyote)

Nombrado por el Coyote de Looney Tunes que corre más allá del borde del acantilado antes de caer. Permite al jugador saltar durante una fracción de segundo (0.1–0.15s) después de abandonar una plataforma. Esto compensa el hecho de que los jugadores frecuentemente presionan saltar un instante después de caer del borde, y sin esta técnica sienten que “el juego se tragó su input”.

Jump Buffering (Buffer de Salto)

Registra la intención de saltar si el botón se presiona ligeramente antes de aterrizar (0.1–0.2s). Cuando el jugador toca el suelo, el salto se ejecuta automáticamente. Sin esto, si el jugador presiona saltar un frame antes de aterrizar, no pasa nada y se siente injusto.

Salto de Altura Variable

La altura del salto depende de cuánto tiempo se mantiene presionado el botón. Al soltar el botón, se reduce la velocidad vertical del personaje (generalmente se multiplica por 0.5), resultando en un salto más corto. Mantener presionado da el salto completo. Esto otorga al jugador control preciso sobre la trayectoria.

Valores recomendados

Coyote Time: 0.1 – 0.15 segundos. Más de 0.2s se siente “floaty”.

Jump Buffer: 0.1 – 0.2 segundos. Suficiente para cubrir imprecisiones humanas.

Salto variable (corte): multiplicar yVelocity por 0.5 al soltar el botón.

Gravedad extra al caer: aplicar un multiplicador de gravedad (2–3x) cuando el personaje cae para que se sienta más pesado y responsivo.

2. Desarrollo Paso a Paso

Esta guía continúa el proyecto **PlatformerGame** de la Guía 5. Abrir el proyecto y la escena Level_01 antes de comenzar.

1 Importar y Configurar Sprite Sheets del Personaje

Preparar las hojas de sprites para las animaciones del personaje:

1.1 Organizar los sprites

5. Copiar las sprite sheets del personaje a Assets/Art/Characters/. Se necesitan al menos:
   - Idle (reposo) — típicamente 4–11 frames
   - Run (correr) — típicamente 6–12 frames
   - Jump (subiendo) — 1–2 frames
   - Fall (cayendo) — 1–2 frames

1.2 Configurar cada sprite sheet

Seleccionar cada imagen en el Inspector y configurar:

Propiedad	Valor
`Sprite Mode`	Multiple
`Pixels Per Unit`	32 (o según el sprite)
`Filter Mode`	Point (no filter)
`Compression`	None

6. Hacer clic en Apply, luego abrir el Sprite Editor.
7. En Sprite Editor: Slice > Type: Grid By Cell Size > configurar el tamaño de cada frame (ej: 32x32).
8. Hacer clic en Slice y luego en Apply.
9. Repetir para cada sprite sheet (Idle, Run, Jump, Fall).

2 Crear los Animation Clips

Crear las animaciones a partir de los sprites cortados:

2.1 Crear el clip de Idle

10. Seleccionar el Player en la Hierarchy.
11. Abrir la ventana Animation: Window > Animation > Animation.
12. Hacer clic en Create en la ventana Animation.
13. Guardar como Player_Idle.anim en Assets/Art/Characters/ (o una carpeta Animations).
14. Expandir el sprite sheet de Idle en la ventana Project (clic en la flecha del asset).
15. Seleccionar todos los frames del Idle y arrastrarlos a la timeline de la ventana Animation.
16. Ajustar la velocidad: en Samples (esquina superior izquierda), configurar entre 8–12 FPS según la fluidez deseada.

2.2 Crear los clips restantes

Repetir el proceso para cada animación:

17. En la ventana Animation, abrir el dropdown del nombre del clip (arriba a la izquierda) y seleccionar Create New Clip.
18. Crear Player_Run.anim — arrastrar los frames de Run, Samples: 12.
19. Crear Player_Jump.anim — arrastrar los frames de Jump, Samples: 8. Desactivar Loop Time en el Inspector del .anim.
20. Crear Player_Fall.anim — arrastrar los frames de Fall, Samples: 8. Desactivar Loop Time.

Loop Time

Idle y Run deben tener Loop Time activado (se repiten continuamente). Jump y Fall deben tener Loop Time desactivado (se reproducen una sola vez y se quedan en el último frame). Para verificar o cambiar, seleccionar el archivo .anim en el Project y revisar en el Inspector.

3 Configurar el Animator Controller

Organizar la máquina de estados para las transiciones entre animaciones:

3.1 Abrir el Animator Controller

21. Al crear el primer clip en el Paso 2, Unity generó automáticamente un Animator Controller (archivo .controller en la misma carpeta).
22. Abrir la ventana Animator: Window > Animation > Animator.
23. Se verán los estados creados: Player_Idle (naranja, estado por defecto), Player_Run, Player_Jump, Player_Fall.

3.2 Crear los parámetros

24. En la ventana Animator, seleccionar la pestaña Parameters.
25. Agregar los siguientes parámetros con el botón +:

Parámetro	Tipo	Uso
`Speed`	Float	Velocidad horizontal
`isGrounded`	Bool	En el suelo
`yVelocity`	Float	Velocidad vertical

3.3 Crear las transiciones

Configurar las transiciones entre estados. Para cada una, hacer clic derecho sobre el estado origen > Make Transition y conectar al estado destino:

26. **Idle → Run:** Condición: Speed > 0.01
27. **Run → Idle:** Condición: Speed < 0.01
28. **Idle → Jump:** Condiciones: isGrounded = false AND yVelocity > 0
29. **Run → Jump:** Condiciones: isGrounded = false AND yVelocity > 0
30. **Jump → Fall:** Condición: yVelocity < 0
31. **Fall → Idle:** Condición: isGrounded = true

3.4 Configurar cada transición

Seleccionar cada flecha de transición y en el Inspector:

32. Desactivar Has Exit Time.
33. Establecer Transition Duration en 0.
34. Verificar que las condiciones sean las correctas.

4 Refactorizar el PlayerController con Movimiento Pulido

Reemplazar **completamente** el script PlayerController.cs de la Guía 5 con esta versión mejorada que incluye Coyote Time, Jump Buffering, salto variable y control de animaciones:

```
C# — PlayerController.cs (versión refactorizada)
using UnityEngine;

public class PlayerController : MonoBehaviour
{
```

```
    [Header("Movement")]
    [SerializeField] private float moveSpeed = 8f;
    [SerializeField] private float acceleration = 60f;
    [SerializeField] private float deceleration = 50f;
    [SerializeField] private float airAcceleration = 40f;

    [Header("Jump")]
    [SerializeField] private float jumpForce = 16f;
    [SerializeField] private float fallMultiplier = 3f;
    [SerializeField] private float lowJumpMultiplier = 5f;

    [Header("Coyote Time & Jump Buffer")]
    [SerializeField] private float coyoteTime = 0.12f;
    [SerializeField] private float jumpBufferTime = 0.15f;

    [Header("Ground Check")]
    [SerializeField] private Transform groundCheck;
    [SerializeField] private float groundCheckRadius = 0.2f;
    [SerializeField] private LayerMask groundLayer;

    // Componentes
    private Rigidbody2D rb;
    private Animator animator;

    // Estado interno
    private float horizontalInput;
    private bool isGrounded;
    private float coyoteTimeCounter;
    private float jumpBufferCounter;
    private bool isJumping;

    // ── IDs de parámetros del Animator (optimización) ──
    private static readonly int SpeedParam =
        Animator.StringToHash("Speed");
    private static readonly int IsGroundedParam =
        Animator.StringToHash("isGrounded");
    private static readonly int YVelocityParam =
        Animator.StringToHash("yVelocity");

    private void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
    }

    private void Update()
    {
        // ── Input ──
        horizontalInput = Input.GetAxisRaw("Horizontal");
```

```
        // ── Ground Check ──
        isGrounded = Physics2D.OverlapCircle(
            groundCheck.position,
            groundCheckRadius,
            groundLayer
        );

        // ── Coyote Time ──
        if (isGrounded)
        {
            coyoteTimeCounter = coyoteTime;
            isJumping = false;
        }
        else
        {
            coyoteTimeCounter -= Time.deltaTime;
        }

        // ── Jump Buffer ──
        if (Input.GetButtonDown("Jump"))
        {
            jumpBufferCounter = jumpBufferTime;
        }
        else
        {
            jumpBufferCounter -= Time.deltaTime;
        }

        // ── Ejecutar salto ──
        if (jumpBufferCounter > 0f && coyoteTimeCounter > 0f
            && !isJumping)
        {
            rb.linearVelocity = new Vector2(
                rb.linearVelocity.x, jumpForce
            );
            isJumping = true;
            jumpBufferCounter = 0f;
            coyoteTimeCounter = 0f;
        }

        // ── Salto variable (soltar = cortar) ──
        if (Input.GetButtonUp("Jump") && rb.linearVelocity.y > 0f)
        {
            rb.linearVelocity = new Vector2(
                rb.linearVelocity.x,
                rb.linearVelocity.y * 0.5f
            );
        }

        // ── Voltear sprite ──
        if (horizontalInput != 0)
```

```
        {
            transform.localScale = new Vector3(
                Mathf.Sign(horizontalInput), 1f, 1f
            );
        }

        // ── Actualizar Animator ──
        animator.SetFloat(SpeedParam,
            Mathf.Abs(horizontalInput));
        animator.SetBool(IsGroundedParam, isGrounded);
        animator.SetFloat(YVelocityParam,
            rb.linearVelocity.y);
    }

    private void FixedUpdate()
    {
        // ── Movimiento horizontal con aceleración ──
        float targetSpeed = horizontalInput * moveSpeed;
        float accelRate = isGrounded
            ? (Mathf.Abs(targetSpeed) > 0.01f
                ? acceleration : deceleration)
            : airAcceleration;

        float speedDiff = targetSpeed - rb.linearVelocity.x;
        float movement = speedDiff * accelRate
            * Time.fixedDeltaTime;

        rb.linearVelocity = new Vector2(
            rb.linearVelocity.x + movement,
            rb.linearVelocity.y
        );

        // ── Gravedad mejorada ──
        if (rb.linearVelocity.y < 0f)
        {
            // Cayendo: gravedad extra
            rb.linearVelocity += Vector2.up
                * (Physics2D.gravity.y * (fallMultiplier - 1f)
                * Time.fixedDeltaTime);
        }
        else if (rb.linearVelocity.y > 0f
            && !Input.GetButton("Jump"))
        {
            // Subiendo pero botón suelto: caer más rápido
            rb.linearVelocity += Vector2.up
                * (Physics2D.gravity.y * (lowJumpMultiplier - 1f)
                * Time.fixedDeltaTime);
        }
    }

    private void OnDrawGizmosSelected()
```

```
    {
        if (groundCheck == null) return;
        Gizmos.color = isGrounded ? Color.green : Color.red;
        Gizmos.DrawWireSphere(
            groundCheck.position, groundCheckRadius
        );
    }
}
```

Animator.StringToHash

Usar Animator.StringToHash() para convertir los nombres de parámetros a IDs enteros es una buena práctica. Es más eficiente que pasar strings en cada frame con SetFloat("Speed", ...), y también previene errores de tipeo silenciosos.

5 Análisis del Código: Qué Cambió y Por Qué

Comparación entre el controlador básico de la Guía 5 y la versión refactorizada:

Aspecto	Guía 5 (Básico)	Guía 6 (Pulido)
Movimiento X	Velocidad directa: horizontal * speed	Con aceleración/desaceleración: lerp suave hacia la velocidad objetivo
Salto	GetButtonDown + isGrounded simple	Coyote Time + Jump Buffer: perdón en los bordes y pre-input
Altura salto	Fija: siempre la misma altura	Variable: soltar botón corta el salto a la mitad
Caída	Gravedad estándar de Unity	Gravedad aumentada al caer: se siente más pesado y preciso
Animaciones	No existían	4 estados con transiciones automáticas por parámetros
Control aéreo	Mismo que en suelo	Aceleración aérea reducida: más compromiso al saltar

Diagrama de flujo del salto

```
Lógica del salto con Coyote Time + Jump Buffer
Update() cada frame:
│
├── ¿En el suelo?
│   ├─ Sí → coyoteTimeCounter = coyoteTime
│   └─ No  → coyoteTimeCounter -= deltaTime
│
```

```
├── ¿Botón Jump presionado?
│   ├─ Sí → jumpBufferCounter = jumpBufferTime
│   └─ No  → jumpBufferCounter -= deltaTime
│
├── ¿jumpBuffer > 0  Y  coyoteTime > 0  Y  !isJumping?
│   └─ Sí → ¡SALTAR! Resetear ambos contadores
│
└── ¿Botón Jump soltado  Y  yVelocity > 0?
    └─ Sí → Cortar velocidad Y a la mitad
```

6 Configurar el Player en el Inspector

Asignar los componentes necesarios al Player:

35. Seleccionar el Player en la Hierarchy.
36. Verificar que tenga el componente Animator (debe haberse agregado automáticamente al crear los clips).
37. Verificar que el Animator Controller esté asignado en el campo Controller del componente Animator.

6.1 Valores recomendados para el Inspector

Campo	Valor Sugerido
`Move Speed`	8
`Acceleration`	60
`Deceleration`	50
`Air Acceleration`	40
`Jump Force`	16
`Fall Multiplier`	3
`Low Jump Multiplier`	5
`Coyote Time`	0.12
`Jump Buffer Time`	0.15
`Ground Check Radius`	0.2
`Ground Layer`	Ground

Experimentar con los valores

Los valores sugeridos son un punto de partida. Cada juego tiene su propia sensación. Se recomienda ajustar moveSpeed, jumpForce, fallMultiplier y los timers de coyote/buffer hasta encontrar el game feel deseado. Cambiar un valor a la vez y probar.

7 Probar la Integración Animaciones + Movimiento

Verificar que todo funcione correctamente:

7.1 Lista de pruebas

38. Presionar Play y verificar cada comportamiento:

Acción	Resultado Esperado
Personaje quieto	Animación Idle reproduciendo en loop
Mover con flechas/AD	Animación Run, sprite se voltea según dirección
Soltar tecla de movimiento	Desaceleración suave (no se detiene instantáneamente)
Presionar Espacio en el suelo	Animación Jump, salto con altura variable
Mantener Espacio	Salto alto completo
Toque rápido de Espacio	Salto corto (aproximadamente la mitad de altura)
Caer del borde de plataforma	Puede saltar durante ~0.12s después (Coyote Time)
Presionar Espacio justo antes de aterrizar	Salto automático al tocar suelo (Jump Buffer)
Caer al vacío	Animación Fall, sensación de gravedad aumentada
Aterrizar	Transición instantánea a Idle o Run según input

7.2 Problemas comunes y soluciones

El personaje no cambia de animación

Verificar que el Animator Controller esté asignado en el componente Animator del Player.

Verificar que los nombres de los parámetros en el Animator coincidan exactamente: Speed, isGrounded, yVelocity (sensible a mayúsculas).

Revisar que Has Exit Time esté desactivado en todas las transiciones.

El personaje salta infinitamente

Verificar que el GroundCheck esté posicionado correctamente (justo debajo de los pies).

Verificar que el Ground Layer esté asignado tanto al Tilemap como al campo del script.

Verificar que la variable isJumping se resetea cuando isGrounded es true.

Las animaciones hacen blending (se mezclan)

Seleccionar cada transición en el Animator y verificar que Transition Duration sea exactamente 0. En pixel art, cualquier blending se ve mal.

3. Ejercicios Propuestos

Ejercicio 1: Animación de Aterrizaje (Landing)

Crear un clip de animación adicional para el momento de aterrizar (landing). Debe ser corto (2–3 frames), reproducirse una sola vez al transicionar de Fall a Idle/Run, y dar feedback visual de que el personaje tocó el suelo. Agregar el estado y las transiciones necesarias al Animator Controller.

Ejercicio 2: Efecto de Squash & Stretch

Implementar un efecto sutil de deformación del sprite: al saltar, estirar ligeramente al personaje en Y (scale Y: 1.1, X: 0.9); al aterrizar, aplastar (scale Y: 0.8, X: 1.2) y regresar al tamaño normal en ~0.1s. Usar una Coroutine o animación para la transición suave.

Ejercicio 3: Indicador Visual de Coyote Time

Para propósitos de depuración y comprensión, agregar un indicador visual temporal (un pequeño círculo o cambio de color del Gizmo) que muestre cuándo el Coyote Time está activo (es decir, cuando el jugador ya dejó la plataforma pero aún puede saltar). Esto ayuda a entender y ajustar el timing.

4. Bibliografía y Recursos

- *Unity Technologies.* Unity Manual — 2D Animation. https://docs.unity3d.com/Manual/class-AnimationClip.html
- *Unity Technologies.* Unity Manual — Animator Controller. https://docs.unity3d.com/Manual/class-AnimatorController.html
- *Unity Technologies.* Unity Manual — Animation Transitions. https://docs.unity3d.com/Manual/class-Transition.html
- *Swink, S.* Game Feel: A Game Designer's Guide to Virtual Sensation. CRC Press, 2008.
- *Pixel Frog.* Pixel Adventure 1 — Free Asset Pack. https://pixelfrog-assets.itch.io/pixel-adventure-1
- *Maddy Thorson & Noel Berry.* Celeste — GDC Talk: “Level Design Workshop” (referencia para Coyote Time y game feel).